
Complete Guide

Alpine Linux Server Setup

for NxrthStack

From bare metal to a fully operational server running a Discord bot, Minecraft server, NAS storage, Cloudflare Tunnels, and Tailscale VPN — all on Alpine Linux.

Services: Discord Bot · Minecraft Server · NAS Storage

Networking: Cloudflare Tunnels · Tailscale VPN

Database: Neon PostgreSQL (shared)

Website: Vercel (nxrthstack.sweber.dev)

OS: Alpine Linux (x86_64)

Table of Contents

Phase 1 Install Alpine Linux (Remove Windows)

- 1.1 Create Bootable USB
- 1.2 Boot from USB
- 1.3 Run Alpine Setup
- 1.4 Enable Community Repository
- 1.5 Create a Non-Root User
- 1.6 Set a Static IP

Phase 2 Install All Dependencies

- 2.1 Essential Packages
- 2.2 Install PM2
- 2.3 Install cloudflared
- 2.4 Install Tailscale

Phase 3 Minecraft Server

- 3.1 Directory Structure
- 3.2 Download Paper MC
- 3.3 Accept EULA & Configure
- 3.4 Create Start Script
- 3.5 OpenRC Service
- 3.6 Test the Server

Phase 4 Discord Bot

- 4.1 Clone & Set Up
- 4.2 Configure Environment
- 4.3 Deploy Slash Commands
- 4.4 PM2 Ecosystem Config
- 4.5 Start the Bot
- 4.6 Verify

Phase 5 NAS Storage Server

- 5.1 Set Up
- 5.2 Configure
- 5.3 Run with PM2
- 5.4 Tailscale Direct HTTPS

Phase 6 Cloudflare Tunnels

- 6.1 Authenticate
- 6.2 Create Tunnel
- 6.3 Configure Multi-Service
- 6.4 DNS Routes
- 6.5 Run as Service
- 6.6 Minecraft Tailscale Approach

Phase 7 Sync — Website ↔ Discord Bot ↔ Discord

- 7.1 How the Sync Works

7.2 Update Vercel Env Vars

7.3 Update Bot .env

7.4 Verify Sync Chain

Phase 8 Firewall & Security

8.1 iptables Configuration

8.2 Automatic Security Updates

Phase 9 Monitoring & Maintenance

9.1 Commands Cheat Sheet

9.2 Install htop

9.3 Log Rotation

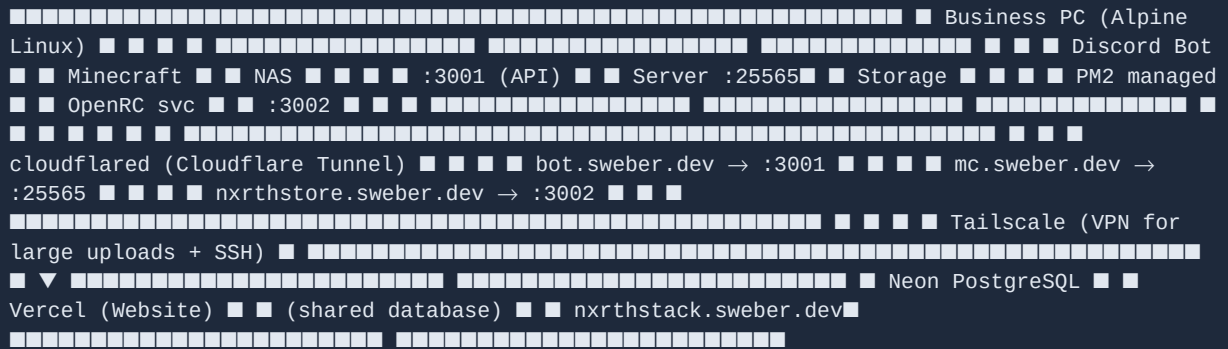
Phase 10 Complete Boot Order Verification

Reference Quick Reference: Service Locations

Appendix Troubleshooting

Architecture Overview

The NxtrStack server runs on a single business PC with Alpine Linux, hosting three main services connected through Cloudflare Tunnels for public access and Tailscale for private VPN access.



PHASE 1 Install Alpine Linux (Remove Windows)

1.1 Create Bootable USB

On your current Windows PC, download Alpine Linux Standard (x86_64) from alpinelinux.org/downloads. Download **alpine-standard-3.21.x-x86_64.iso**.

Flash it to USB with Rufus:

1. Download Rufus from <https://rufus.ie>
2. Insert USB drive (min 2GB)
3. Select the Alpine ISO
4. Partition scheme: GPT
5. Target system: UEFI
6. Click Start

1.2 Boot from USB

1. Plug USB into the new business PC
2. Power on → press F2/F12/DEL (depends on BIOS) to enter boot menu
3. Select the USB drive (UEFI mode)
4. You'll see "localhost login:" → type: **root** (no password)

1.3 Run Alpine Setup

```
setup-alpine
```

Answer the prompts as follows:

Prompt	Answer	Explanation
Keyboard layout	de	German layout (change if needed)
Variant	de	
Hostname	nrxthserver	Your server's name
Network interface	eth0 (or shown interface)	Wired ethernet
IP address	dhcp	Auto from router (static later)
Manual network config	n	
Root password	Set a strong one	Remember this!
Timezone	Europe/Berlin	
Proxy	none	

Prompt	Answer	Explanation
NTP client	chrony	Keeps time accurate
Mirror	1 (or f for fastest)	Package repository
SSH server	openssh	Remote access
Disk to use	sda (your main disk)	This wipes Windows!
How to use disk	sys	Full system install
Erase disk	y	Confirms Windows removal

After it finishes:

```
# Remove USB and reboot reboot
```

1.4 Post-Install: Enable Community Repository

Log in as root:

```
# Edit repositories to enable community packages vi /etc/apk/repositories
```

Uncomment (remove the #) the community line so it looks like:

```
https://dl-cdn.alpinelinux.org/alpine/v3.21/main
https://dl-cdn.alpinelinux.org/alpine/v3.21/community
```

Save: press **Esc**, type **:wq**, press **Enter**.

```
# Update package index apk update && apk upgrade
```

1.5 Create a Non-Root User

```
# Create your user with home directory and bash shell apk add bash sudo shadow adduser -s
/bin/bash sweber # Set password when prompted # Add to sudo group addgroup sweber wheel #
Enable wheel group for sudo visudo # Uncomment this line: %wheel ALL=(ALL:ALL) ALL # Switch to
your user for the rest of the guide su - sweber
```

1.6 Set a Static IP (Optional but Recommended)

This prevents the server IP from changing after router restarts.

```
sudo vi /etc/network/interfaces
```

Replace the eth0 block:

```
auto eth0 iface eth0 inet static address 192.168.1.100 # Pick an IP outside your router's DHCP
range netmask 255.255.255.0 gateway 192.168.1.1 # Your router IP dns-nameservers 1.1.1.1
1.0.0.1
```

```
sudo rc-service networking restart
```

PHASE 2 Install All Dependencies

2.1 Essential Packages

```
sudo apk add \ curl wget git \ nodejs npm \ openjdk21-jre \ screen \ openrc \ ca-certificates \
build-base \ python3
```

Package	Why
nodejs npm	Discord bot + NAS storage server
openjdk21-jre	Minecraft server (Java runtime)
screen	Run processes in detachable terminals
build-base	Compile native Node modules (bcrypt etc.)
python3	Required by some npm packages (node-gyp)

2.2 Install PM2 (Process Manager)

```
# Install PM2 globally for managing the bot sudo npm install -g pm2 # Install tsx globally for
TypeScript execution sudo npm install -g tsx
```

2.3 Install cloudflared

```
# Download cloudflared for Linux amd64 sudo wget -O /usr/local/bin/cloudflared \
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64
sudo chmod +x /usr/local/bin/cloudflared # Verify cloudflared --version
```

2.4 Install Tailscale

```
# Alpine has Tailscale in community repo sudo apk add tailscale # Enable and start Tailscale
daemon sudo rc-update add tailscale default sudo rc-service tailscale start # Authenticate
(opens a URL you paste in browser) sudo tailscale up --ssh # Copy the URL shown, open it in a
browser, and log in
```

The `--ssh` flag enables Tailscale SSH so you can SSH into this machine from anywhere on your tailnet without managing SSH keys.

PHASE 3 Minecraft Server

3.1 Create Directory Structure

```
sudo mkdir -p /opt/minecraft sudo chown sweber:sweber /opt/minecraft cd /opt/minecraft
```

3.2 Download Paper MC (Recommended over Vanilla)

Paper is a high-performance fork with plugin support. Check papermc.io/downloads/paper for the latest build.

```
# Download Paper 1.21.4 (check for latest version) wget -O paper.jar \ "https://api.papermc.io/v2/projects/paper/versions/1.21.4/builds/224/downloads/paper-1.21.4-224.jar"
```

3.3 Accept EULA and Configure

```
# Create eula.txt (required to start) echo "eula=true" > eula.txt
```

Create `server.properties`:

```
cat > server.properties << 'EOF' # Server Settings server-port=25565 motd=$b$lnXrthStack
$7Minecraft Server max-players=20 difficulty=normal gamemode=survival view-distance=12
simulation-distance=8 online-mode=true white-list=false enable-command-block=true
spawn-protection=0 # Performance max-tick-time=60000 network-compression-threshold=256 # RCON
(Remote Console) enable-rcon=true rcon.port=25575
rcon.password=CHANGE_THIS_TO_A_STRONG_PASSWORD EOF
```

3.4 Create Start Script

```
cat > start.sh << 'EOF' #!/bin/bash # Adjust -Xms (min) and -Xmx (max) to your available RAM #
Leave ~2-4GB for the OS + other services java -Xms4G -Xmx8G \ -XX:+UseG1GC \
-XX:+ParallelRefProcEnabled \ -XX:MaxGCPauseMillis=200 \ -XX:+UnlockExperimentalVMOptions \
-XX:+DisableExplicitGC \ -XX:G1NewSizePercent=30 \ -XX:G1MaxNewSizePercent=40 \
-XX:G1HeapRegionSize=8M \ -XX:G1ReservePercent=20 \ -XX:G1MixedGCCountTarget=4 \
-XX:InitiatingHeapOccupancyPercent=15 \ -XX:G1MixedGCLiveThresholdPercent=90 \
-XX:G1RSSHGCLiveThresholdPercent=50 \ -XX:SurvivorRatio=32 \ -XX:+PerfDisableSharedMem \
-XX:MaxTenuringThreshold=1 \ -jar paper.jar --nogui EOF chmod +x start.sh
```

3.5 Create OpenRC Service (Auto-Start on Boot)

```
sudo cat > /etc/init.d/minecraft << 'SVCEOF' #!/sbin/openrc-run name="Minecraft Server"
description="Paper Minecraft Server" command="/bin/bash"
command_args="/opt/minecraft/start.sh" command_user="sweber" directory="/opt/minecraft"
pidfile="/run/minecraft.pid" command_background="yes"
output_log="/opt/minecraft/logs/service.log" error_log="/opt/minecraft/logs/service-error.log"
depend() { need net after firewall } start_pre() { checkpath -d -o sweber:sweber
/opt/minecraft/logs } SVCEOF sudo chmod +x /etc/init.d/minecraft sudo rc-update add minecraft
default
```

3.6 Test the Minecraft Server

```
cd /opt/minecraft ./start.sh # Wait for "Done!" message, then type 'stop' to shut it down # Now  
start it as a service: sudo rc-service minecraft start
```

PHASE 4 Discord Bot

4.1 Clone and Set Up

```
sudo mkdir -p /opt/nxrthstack sudo chown sweber:sweber /opt/nxrthstack cd /opt/nxrthstack #  
Clone your repo (or copy from your dev machine) git clone  
https://github.com/YOUR_USERNAME/nxrthstack.git repo cd repo/nxrthstack-bot # Install  
dependencies npm install
```

4.2 Configure Environment

```
cat > .env << 'EOF' DISCORD_BOT_TOKEN=your_bot_token_from_discord_developer_portal  
DISCORD_CLIENT_ID=1468315439309262981 DISCORD_CLIENT_SECRET=your_client_secret  
DISCORD_GUILD_ID=your_discord_server_id  
DATABASE_URL=postgresql://neondb_owner:YOUR_PASSWORD@ep-proud-surf-...  
NXRTH_API_URL=https://nxrthstack.sweber.dev API_PORT=3001 EOF # Lock down permissions (only  
you can read this) chmod 600 .env
```

4.3 Deploy Slash Commands

```
# Register all slash commands with Discord API npx tsx src/deploy-commands.ts
```

4.4 PM2 Ecosystem Config

The bot already has an `ecosystem.config.cjs`. Update paths for Linux:

```
cat > ecosystem.config.cjs << 'EOF' module.exports = { apps: [{ name: 'nxrthstack-bot', script:  
'src/index.ts', interpreter: 'tsx', cwd: '/opt/nxrthstack/repo/nxrthstack-bot', env: {  
NODE_ENV: 'production' }, max_restarts: 10, min_uptime: '10s', restart_delay: 5000, error_file:  
'/opt/nxrthstack/logs/bot-error.log', out_file: '/opt/nxrthstack/logs/bot-out.log',  
log_date_format: 'YYYY-MM-DD HH:mm:ss', watch: false, }] } EOF mkdir -p /opt/nxrthstack/logs
```

4.5 Start the Bot

```
cd /opt/nxrthstack/repo/nxrthstack-bot # Start with PM2 pm2 start ecosystem.config.cjs # Save  
PM2 process list so it restarts on reboot pm2 save # Set up PM2 to start on boot (generates an  
OpenRC startup script) pm2 startup openrc -u sweber --hp /home/sweber # Run the command it  
outputs (sudo env PATH=... pm2 startup ...)
```

4.6 Verify

```
pm2 status # Should show 'nxrthstack-bot' as 'online' pm2 logs # Watch live logs curl  
localhost:3001 # Should respond (bot webhook API)
```

PHASE 5 NAS Storage Server

5.1 Set Up

```
cd /opt/nxrthstack/repo/nas-storage-server npm install # Create storage directory sudo mkdir -p /data/clips sudo chown sweber:sweber /data/clips
```

5.2 Configure

```
cat > .env << 'EOF' PORT=3002 NAS_API_KEY=$(openssl rand -base64 32) STORAGE_PATH=/data/clips PUBLIC_URL=https://nxrthstore.sweber.dev MAX_FILE_SIZE=5368709120 ALLOWED_ORIGINS=http://localhost:3000,https://nxrthstack.sweber.dev HTTPS_PORT=3443 TLS_CERT_PATH=/opt/nas-storage-server/tls.crt TLS_KEY_PATH=/opt/nas-storage-server/tls.key EOF chmod 600 .env
```

Important: Copy the generated NAS_API_KEY value — you'll need it in your Vercel environment variables for the website to upload files.

Using port **3002** here since the Discord bot already uses 3001.

5.3 Run with PM2

```
# Add to PM2 pm2 start server.js --name nas-storage --cwd /opt/nxrthstack/repo/nas-storage-server # Save updated process list pm2 save
```

5.4 Set Up Tailscale Direct HTTPS (for >100MB Uploads)

```
# Get Tailscale hostname tailscale status # Note your hostname, e.g. nxrthserver # Generate TLS certs via Tailscale sudo tailscale cert nxrthserver.tailnet-name.ts.net # Copy certs to storage server location sudo mkdir -p /opt/nas-storage-server sudo cp nxrthserver.tailnet-name.ts.net.crt /opt/nas-storage-server/tls.crt sudo cp nxrthserver.tailnet-name.ts.net.key /opt/nas-storage-server/tls.key sudo chown sweber:sweber /opt/nas-storage-server/tls.*
```

Update the .env with your actual Tailscale hostname:

```
echo "NAS_TAILSCALE_URL=https://nxrthserver.tailnet-name.ts.net:3443" >> \ /opt/nxrthstack/repo/nas-storage-server/.env
```

PHASE 6 Cloudflare Tunnels

This is the critical networking piece — it exposes your local services to the internet without port forwarding.

6.1 Authenticate cloudflared

```
cloudflared tunnel login # Opens a URL – paste it in your browser and select your Cloudflare zone (sweber.dev)
```

6.2 Create the Tunnel

```
# Create a named tunnel cloudflared tunnel create nxrthserver # This outputs a Tunnel ID (like a1b2c3d4-...) # and creates a credentials file at ~/.cloudflared/<TUNNEL_ID>.json
```

6.3 Configure the Tunnel (Multi-Service)

```
mkdir -p ~/.cloudflared cat > ~/.cloudflared/config.yml << 'EOF' tunnel: YOUR_TUNNEL_ID
credentials-file: /home/sweber/.cloudflared/YOUR_TUNNEL_ID.json ingress: # Discord Bot Webhook
API - hostname: bot.sweber.dev service: http://localhost:3001 # NAS Storage Server (clips,
uploads <100MB via CF) - hostname: nxrthstore.sweber.dev service: http://localhost:3002
originRequest: httpHostHeader: nxrthstore.sweber.dev # Catch-all (required) - service:
http_status:404 EOF
```

Replace **YOUR_TUNNEL_ID** with the actual ID from step 6.2. Cloudflare Tunnels do NOT support raw TCP on free plans — Minecraft uses Tailscale or port forwarding instead.

6.4 Create DNS Routes

```
# Route bot.sweber.dev → tunnel cloudflared tunnel route dns nxrthserver bot.sweber.dev #
Route nxrthstore.sweber.dev → tunnel cloudflared tunnel route dns nxrthserver
nxrthstore.sweber.dev
```

6.5 Run Tunnel as a Service

```
sudo cat > /etc/init.d/cloudflared << 'SVCEOF' #!/sbin/openrc-run name="Cloudflare Tunnel"
description="cloudflared tunnel daemon" command="/usr/local/bin/cloudflared"
command_args="tunnel --config /home/sweber/.cloudflared/config.yml run" command_user="sweber"
pidfile="/run/cloudflared.pid" command_background="yes" output_log="/var/log/cloudflared.log"
error_log="/var/log/cloudflared-error.log" depend() { need net } SVCEOF sudo chmod +x
/etc/init.d/cloudflared sudo rc-update add cloudflared default sudo rc-service cloudflared
start
```

6.6 Minecraft: Tailscale Approach (Free, No Port Forwarding)

Option A — Tailscale (Private, invite players to tailnet): Players install Tailscale, you share access. They connect to: **nxrthserver:25565**.

Option B — Port Forwarding (Public, traditional): On your router, forward external port 25565 → 192.168.1.100:25565 (TCP). Players connect to your public IP or mc.sweber.dev.

For Option B, create a DNS A record in Cloudflare: Type: A, Name: mc, Content: YOUR_PUBLIC_IP, Proxy: OFF (gray cloud — DNS only).

PHASE 7 Sync — Website ↔ Discord Bot ↔ Discord

7.1 How the Sync Already Works

Your existing architecture has this flow:

```
Website (Vercel) ■■■webhook POST■■■■ Bot API (bot.sweber.dev:3001) ■■■■ Discord Channels ■■■■  
■■■■ shared Neon PostgreSQL ■■■■
```

1. **Website** → **Bot**: When events happen (achievement unlock, session created, rivalry match, announcement), the website POSTs to `DISCORD_BOT_API_URL/webhook`.
2. **Bot** → **Discord**: The bot receives the webhook and broadcasts embeds to configured channels.
3. **Both** → **Database**: Both read/write the same Neon PostgreSQL database.

7.2 Update Vercel Environment Variables

In your Vercel dashboard (nxrthstack project → Settings → Environment Variables), update:

```
DISCORD_BOT_API_URL=https://bot.sweber.dev NAS_STORAGE_URL=https://nxrthstore.sweber.dev  
NAS_TAILSCALE_URL=https://nxrthserver.tailnet-name.ts.net:3443 NAS_API_KEY=<the key you  
generated in Phase 5>
```

7.3 Update Bot .env for Self-Reference

```
# Add to /opt/nxrthstack/repo/nxrthstack-bot/.env echo "BOT_PUBLIC_URL=https://bot.sweber.dev"  
>> /opt/nxrthstack/repo/nxrthstack-bot/.env
```

7.4 Verify the Full Sync Chain

```
# 1. Test bot API is reachable through tunnel curl https://bot.sweber.dev # 2. Test NAS storage  
through tunnel curl https://nxrthstore.sweber.dev # 3. Test bot can reach database pm2 logs  
nxrthstack-bot --lines 20 # 4. Test webhook from website → bot curl -X POST  
https://bot.sweber.dev/webhook \ -H "Content-Type: application/json" \ -H "Authorization:  
Bearer YOUR_DISCORD_CLIENT_SECRET" \ -d '{"type": "announcement.created", "data":  
{"announcementId": "test"}}'
```

PHASE 8 Firewall & Security

8.1 Install and Configure iptables

```
sudo apk add iptables ip6tables # Allow loopback sudo iptables -A INPUT -i lo -j ACCEPT # Allow established connections sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT # Allow SSH (from any – or restrict to your IP) sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # Allow Tailscale sudo iptables -A INPUT -i tailscale0 -j ACCEPT # Allow Minecraft (only if using port forwarding, Option B) sudo iptables -A INPUT -p tcp --dport 25565 -j ACCEPT # Drop everything else sudo iptables -A INPUT -j DROP # Save rules to persist across reboots sudo rc-update add iptables default sudo /etc/init.d/iptables save
```

Note: cloudflared makes outbound connections (not inbound), so no ports need to be opened for the tunnel itself.

8.2 Automatic Security Updates

```
# Add a daily cron job for security updates sudo cat > /etc/periodic/daily/apk-upgrade << 'EOF'
#!/bin/sh apk update && apk upgrade --no-cache EOF sudo chmod +x
/etc/periodic/daily/apk-upgrade
```


PHASE 9 Monitoring & Maintenance

9.1 Useful Commands Cheat Sheet

Command	What it does
<code>pm2 status</code>	Show all PM2 processes (bot + NAS)
<code>pm2 logs</code>	Live logs from all PM2 apps
<code>pm2 restart all</code>	Restart bot + NAS
<code>sudo rc-service minecraft status</code>	Check Minecraft server
<code>sudo rc-service minecraft restart</code>	Restart Minecraft
<code>screen -r minecraft</code>	Attach to MC console (if using screen)
<code>cloudflared tunnel info</code>	Tunnel status
<code>tailscale status</code>	Tailscale network status
<code>htop</code>	System resource monitor
<code>sudo rc-status</code>	All running services

9.2 Install htop for Monitoring

```
sudo apk add htop
```

9.3 Log Rotation

```
sudo apk add logrotate sudo cat > /etc/logrotate.d/nxrthstack << 'EOF'
/opt/nxrthstack/logs/*.log { daily rotate 7 compress missingok notifempty }
/opt/minecraft/logs/*.log { daily rotate 7 compress missingok notifempty }
/var/log/cloudflared*.log { daily rotate 7 compress missingok notifempty } EOF
```

PHASE 10 Complete Boot Order Verification

After everything is set up, reboot and verify all services start automatically:

```
sudo reboot
```

After reboot, SSH back in (via Tailscale SSH or local):

```
# Check all services sudo rc-status # OpenRC services (minecraft, cloudflared, tailscale) pm2 status # PM2 processes (bot, NAS storage) curl localhost:3001 # Bot API curl localhost:3002 # NAS storage cloudflared tunnel info # Tunnel connected tailscale status # Tailscale connected
```

Quick Reference: Service Locations

Service	Location	Port	Managed By	Public URL
Discord Bot	/opt/nxrthstack/repo/nxrthstack-bot	3001	PM2	bot.sweber.dev
NAS Storage	/opt/nxrthstack/repo/nas-storage-server	3002	PM2	nxrthstore.sweber.dev
Minecraft	/opt/minecraft	25565	OpenRC	mc.sweber.dev (DNS only)
CF Tunnel	~/.cloudflared/config.yml	—	OpenRC	—
Tailscale	system	—	OpenRC	nxrthserver.ts.net

Troubleshooting

Problem	Fix
Bot can't reach DB	Check DATABASE_URL in .env, verify nslookup works: apk add bind-tools && nslookup <host>
Cloudflare tunnel 502	Service isn't running on expected port – check pm2 status and port numbers in config.yml
Minecraft won't start	Check Java: java -version, check RAM flags in start.sh aren't exceeding physical RAM
PM2 not starting on boot	Re-run pm2 startup openrc -u sweber --hp /home/sweber and execute the generated command

Problem	Fix
Website webhooks fail	Verify DISCORD_BOT_API_URL in Vercel env vars points to <code>https://bot.sweber.dev</code>
Large upload fails via CF	Use Tailscale direct URL — CF free plan limits uploads to ~100MB

*This covers the complete pipeline from bare metal to everything running.
Work through it phase by phase — each phase is independent enough that you can verify it works before moving to the next.*